

DOCUMENTAÇÃO TÉCNICA COMPLEMENTAR: SISTEMA TODO-LIST FULL STACK

Objetivo: Este documento serve como especificação técnica formal e manual de apoio para a apresentação acadêmica do projeto Todo-List. Ele detalha a infraestrutura de gerenciamento, a arquitetura do ecossistema móvel e o funcionamento lógico do servidor de aplicação.

1. CONCEITOS E INFRAESTRUTURA DE GERENCIAMENTO DE PACOTES

O desenvolvimento moderno em ecossistemas JavaScript baseia-se fortemente na modularização. Para isso, são utilizadas as seguintes ferramentas básicas:

- **npm (Node Package Manager):** O gerenciador de pacotes padrão do ecossistema Node.js. É encarregado de baixar, catalogar e gerenciar o ciclo de vida das dependências externas declaradas no projeto.
- **npx (Node Package Execute):** Um executor de pacotes integrado ao npm. Permite baixar temporariamente e executar ferramentas de linha de comando sem a obrigatoriedade de uma instalação global ou local permanente no sistema operacional.
- **Dependência:** Bibliotecas ou blocos de código externos concebidos por terceiros que estendem as funcionalidades do projeto. O uso de dependências mitiga o retrabalho, acelerando o desenvolvimento por meio de soluções previamente validadas pela comunidade.

2. ECOSSISTEMA DO APLICATIVO MÓVEL (FRONTEND REACT NATIVE & EXPO)

A arquitetura do aplicativo mobile baseia-se em quatro pilares fundamentais: persistência local de dados, comunicação de rede, roteamento entre telas e consistência de ambiente. Os pacotes instalados resolvem especificamente essas frentes:

2.1. Comandos de Instalação e Análise de Dependências

```
npm install @react-native-async-storage/async-storage axios
npm install expo
npm install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context
npx expo doctor-fix-dependencies
```

2.2. Detalhamento de Componentes Tecnológicos Móvel

- **@react-native-async-storage/async-storage:** Mecanismo de persistência de dados local, assíncrono e do tipo chave-valor (não-relacional). Alocado diretamente no armazenamento interno do dispositivo do usuário, ele é encarregado de reter estados que precisam sobreviver ao fechamento da aplicação, como tokens de autenticação ou preferências do sistema.
- **axios:** Cliente HTTP baseado em Promises, responsável por gerenciar as requisições para servidores externos. Funciona como a ponte de comunicação que consome a API RESTful do backend, enviando dados e interpretando respostas JSON.
- **expo:** Framework e ecossistema unificado construído sobre o React Native. Ele provê abstração e acesso simplificado às APIs nativas de hardware, agilizando fluxos de compilação, testes em tempo real e o processo de build do aplicativo.
- **@react-navigation/native e @react-navigation/native-stack:** Juntos, estruturam o fluxo de navegação do app. O módulo nativo gerencia a árvore e o histórico global de telas, enquanto o módulo *stack* implementa a navegação em formato de "pilha", sobrepondo novas interfaces à tela anterior e estruturando o histórico de retorno.
- **react-native-screens:** Otimizador de performance. Garante que telas que estejam em segundo plano ou fora da visualização atual do usuário não consumam ciclos de processamento e memória do dispositivo.

- **react-native-safe-area-context:** Componente de conformidade de layout. Ele detecta os limites físicos da tela do aparelho, calculando espaçamentos para impedir que o conteúdo da interface fique sob "notches", sensores de câmera, barras de status ou barras de navegação do iOS e Android.
- **npx expo doctor fix-dependencies:** Utilitário de diagnóstico e correção. O parâmetro realiza uma varredura no arquivo package.json, avaliando se há incompatibilidades de versões entre as dependências e o núcleo do Expo, ajustando-as automaticamente para mitigar bugs de compilação.

2.3. Arquivo de Compilação: babel.config.js

O Babel funciona como um transpilador de código. Ele converte sintaxes modernas do JavaScript moderno (como *arrow functions*, *optional chaining* e a estrutura JSX do React) em código ES5, garantindo ampla retrocompatibilidade com motores Javascript mais antigos de smartphones.

```
module.exports = function(api) {  
  api.cache(true);  
  return {  
    presets: ['babel-preset-expo'],  
  };  
};
```

O preset babel-preset-expo traz um compilado de regras configurado nativamente pela equipe do Expo, abstraindo a tradução de JSX e TypeScript para ambientes Android e iOS. Esse arquivo pode ser editado futuramente para inclusão de plugins como mapeamento de apelidos de pastas (*Module Resolver*), injeção de variáveis de ambiente e suporte a animações complexas.

3. ECOSSISTEMA DO SERVIDOR (BACKEND PYTHON & FLASK)

O backend funciona como uma API RESTful centralizada que processa as regras de negócio e intermedia o acesso ao banco de dados.

3.1. Infraestrutura do Servidor

```
py -m pip install flask flask-cors PyJWT pymysql
```

O prefixo `py -m pip` garante o uso do gerenciador de pacotes vinculado de forma estrita à instância correta do interpretador Python no Windows, mitigando conflitos globais de ambiente.

3.2. Detalhamento dos Módulos do Backend

- **flask:** Micro-framework focado na estruturação de rotas de rede e gerenciamento de requisições web HTTP através de mapeamentos de endpoints lógicos.
- **flask-cors:** Módulo gerenciador de políticas de CORS (Cross-Origin Resource Sharing). Essencial para configurar os cabeçalhos HTTP que autorizam explicitamente o aplicativo front-end (Expo/React) a enviar requisições e consumir os recursos lógicos do servidor sem sofrer bloqueios de segurança originários dos navegadores ou do sistema operacional.
- **PyJWT:** Biblioteca responsável pela geração, decodificação e validação de JSON Web Tokens (JWT). Provê segurança sem estado (stateless) para a aplicação: após a validação das credenciais do usuário, um token encriptado é gerado e repassado ao front-end para que as próximas requisições comprovem a identidade do usuário.
- **pymysql:** Driver/Conector de banco de dados baseado puramente em Python. Ele estabelece o canal de comunicação direto entre as funções da aplicação Flask e o servidor relacional MySQL, permitindo a execução de queries SQL como instruções de persistência (INSERT, SELECT, UPDATE, DELETE).

4. ARQUITETURA DA API E SEGURANÇA NO CRUD

Abaixo são apresentados os pilares arquiteturais do código e a estrutura lógica dos endpoints disponíveis.

4.1. Integração CORS de Amplo Espectro

```
CORS(app, resources={r"/*": {"origins": "*"}})
```

Roteiro para a Apresentação: "Para permitir que o nosso aplicativo frontend (como o React) consiga consumir esta API sem problemas de bloqueio de segurança do navegador, nós configuramos o CORS para liberar o acesso a todas as rotas."

4.2. Fluxo de Autenticação e Isolamento Multitenancy

A segurança do sistema reside na função obter_usuario_id(). Ela atua como um interceptador central em requisições privadas:

- 1. Extrai o cabeçalho HTTP 'Authorization' da requisição.
- 2. Isola o token eliminando o prefixo 'Bearer '.
- 3. Usa a SECRET_KEY interna para decodificar o payload e extrair o usuario_id.

Esse usuario_id é obrigatoriamente acoplado a qualquer comando de manipulação de dados (CRUD) no banco de dados MySQL, prevenindo vulnerabilidades de Broken Object Level Authorization (BOLA):

```
cursor.execute("SELECT id FROM tarefas WHERE id = %s AND usuario_id = %s", (id, usuario
```

Roteiro para a Apresentação: "A nossa API conta com uma camada rigorosa de proteção de dados. Um usuário logado jamais conseguirá ver, atualizar ou deletar a tarefa de outro usuário, porque todas as consultas SQL validam o id da tarefa junto com o usuario_id extraído do Token JWT."

4.3. Resiliência de Conexões de Banco de Dados

A persistência foi programada sob blocos de tratamento de exceção estruturados:

```
try:
    # Execução de comandos SQL (Queries)
finally:
    db.close()
```

Roteiro para a Apresentação: "Garantimos a resiliência do backend usando a estrutura try...finally. Mesmo se ocorrer algum erro inesperado no banco de dados, a conexão será fechada obrigatoriamente no final, evitando vazamento de memória e travamento do servidor."

4.4. Matriz de Endpoints da API (Rotas)

Método HTTP	Rota (Endpoint)	Função Principal	Requer Autenticação (JWT)?
POST	/cadastro	Cadastra um novo usuário no sistema, aplicando validações para evitar e-mails duplicados.	Não
POST	/login	Valida as credenciais enviadas e gera um Token JWT válido com expiração programada.	Não

Método HTTP	Rota (Endpoint)	Função Principal	Requer Autenticação (JWT)?
GET	<code>/tarefas</code>	Retorna a lista de tarefas associadas exclusivamente ao usuário logado.	Sim
POST	<code>/tarefas</code>	Insere uma nova tarefa no banco de dados associando-a ao ID do usuário autenticado.	Sim
PUT	<code>/tarefas/<id></code>	Atualiza propriedades específicas da tarefa (título ou status de conclusão) baseado no ID.	Sim
DELETE	<code>/tarefas/<id></code>	Remove de forma física e segura um registro de tarefa do banco de dados MySQL.	Sim

5. DIRETRIZES DE DEFESA CONTRA A BANCA EXAMINADORA

Preparação antecipada para questionamentos técnicos previsíveis sobre o estado do código:

- Questionamento:** *"Por que as senhas estão sendo armazenadas em texto claro (plaintext) no banco de dados sem passar por processos de criptografia?"*
Resposta Estratégica: "Como este projeto possui escopo estritamente acadêmico, o foco principal do desenvolvimento residiu na validação do fluxo do protocolo JWT e na modelagem das rotas. Para um ambiente de produção comercial, a evolução imediata e obrigatória do software consistiria no emprego de bibliotecas como `bcrypt` ou o módulo `werkzeug.security` para aplicar funções de hash criptográfico (como o PBKDF2) com 'salts' aleatórios antes do armazenamento."
- Questionamento:** *"A propriedade SECRET_KEY está declarada diretamente no código fonte (Hardcoded). Qual o risco e como mitigar?"*
Resposta Estratégica: "A exposição estática da chave privada ocorreu de forma intencional exclusivamente para agilizar o ambiente de testes local e a demonstração perante a banca. Em conformidade com os pilares de segurança e governança de software, o padrão definitivo exige a migração desse segredo para um arquivo isolado de variáveis de ambiente (como um arquivo `.env`), blindando o código e evitando o vazamento acidental de chaves de criptografia para repositórios públicos do GitHub."